

Detailed Implementation and Evaluation Report

Native-Resolution Pneumonia Classification with a Frozen Image Encoder and
Four-Qubit Quantum Fidelity Attention

Updated project documentation
Source, checkpoints, and verification evidence retained in **UPDATED**

2 October 2026

Abstract

This report documents the completed version of the project that exceeds 90% test accuracy while retaining quantum-circuit attention. Official native 224-pixel PneumoniaMNIST images are processed by a frozen ImageNet-pretrained ResNet-18. Four spatial feature tokens are projected into a small transformer whose query-key similarities are four-qubit state fidelities. The entangled feature map uses angle encoding, a CNOT chain, and a second data-encoding stage. A product-state circuit and a parameter-matched classical dot-attention model provide controls. Training uses AdamW, training-only feature normalization and flip views, feature MixUp, exponential moving average parameters, and validation-based checkpoint selection and temperature calibration.

Across five seeds, entangled quantum attention achieves $91.60\% \pm 0.58\%$ single-model accuracy and 91.67% ensemble accuracy (572 of 624 test images). Product quantum attention achieves 91.19% ensemble accuracy; the matched classical dot ensemble achieves 92.15%. Independent PennyLane compute-uncompute inference reproduces every quantum classifier's decisions on the complete test split. These results establish a working, reproducible hybrid classifier, but do not demonstrate quantum superiority, quantum speedup, algorithmic novelty, or clinical readiness. The official test cohort had been examined in earlier experiments, so this study is exploratory. Native resolution and external pretraining change the comparison regime relative to the earlier 88.94% result.

Contents

1	Purpose, scope, and the meaning of the result	5
1.1	The problem being solved	5
1.2	What has actually been achieved	5
1.3	What this result cannot establish	5
2	Data and preprocessing	7
2.1	Official data and split roles	7
2.2	Why native resolution matters	7
2.3	Image transformations	8
2.4	Training-only feature normalization	8
3	The classical image encoder and transformer	9
3.1	Frozen ResNet-18 transfer learning	9
3.2	Token projection and sequence construction	9
3.3	Two pre-normalized transformer blocks	10
4	Quantum algorithms: gates, encoding, and fidelity	11
4.1	Qubits and the encoded state	11
4.2	Rotation gates	11
4.3	The product-state feature map	11
4.4	The entangled, data-reuploading feature map	12
4.5	Why the second encoding stage is important	12
4.6	Compute–uncompute fidelity estimation	13
4.7	Earlier SWAP-test algorithms and their relation to this version	13
5	How quantum similarities become attention	14
5.1	Queries, keys, and values	14
5.2	Softmax and aggregation	14
5.3	The parameter-matched dot control	14
5.4	Gradients and simulation	15
6	Training and model selection, step by step	16
6.1	Fixed settings	16
6.2	Class-weighted cross-entropy	16
6.3	Feature MixUp	17
6.4	AdamW and the learning-rate schedule	17
6.5	Exponential moving average and checkpoint selection	17
6.6	Temperature calibration and ensemble construction	18
6.7	Resumability and reproducibility	18
6.8	Training algorithm	18

7	Controls, evaluation, and measured results	19
7.1	Classical linear control	19
7.2	Metrics and their interpretation	19
7.3	Main accuracy and AUROC results	19
7.4	Paired seed statistics	20
7.5	Paired ensemble comparisons	21
8	Independent verification of the quantum path	22
8.1	What is checked	22
8.2	Complete PennyLane inference	22
8.3	Tests and provenance	22
9	Earlier attempts and why they remain relevant	24
9.1	The older 88.94% reference	24
9.2	Other unsuccessful branches	24
9.3	What is plausible, and what is measured	25
10	Reproduction and the UPDATED package	26
10.1	Directory map	26
10.2	Running the retained study	26
10.3	Preparing or training a new study	27
10.4	GitHub distribution and asset handling	27
11	Contribution, limitations, and defensible presentation	28
11.1	A precise contribution statement	28
11.2	Publication and novelty	28
11.3	Remaining limitations	28
A	Implementation source used for the result	30
A.1	Quantum gates and attention	30
A.2	Pretrained encoder and transfer head	33
A.3	Training, selection, and linear control	34
A.4	Native-resolution adapter and correspondence checks	37
A.5	Frozen experiment configuration	39

List of Figures

3.1	The complete hybrid inference pipeline. The quantum component supplies similarities; image encoding and value processing are classical.	10
7.1	Retained comparison figure: single-model mean and standard deviation, ensemble outcomes, and classical linear control.	20

List of Tables

2.1	Official split counts. Training supplies gradients, validation supplies selection and calibration, and test supplies evaluation.	7
6.1	Frozen training recipe shared by all neural kernels.	16
7.1	Five-seed neural results and one deterministic linear control. SD describes variation over training seeds on the same test cohort.	19
7.2	Every quantum seed exceeds 90%; no failed seed is removed. Rounded values are derived from the retained per-seed CSV.	20
7.3	Exploratory paired-seed effects. All associated Holm-adjusted p values equal 1.0.	21
9.1	Selected earlier outcomes and the final resolution branch. These are separate regimes rather than a single controlled treatment comparison.	24

Chapter 1

Purpose, scope, and the meaning of the result

1.1 The problem being solved

The task is binary classification of chest radiograph images in PneumoniaMNIST: label 0 denotes normal and label 1 denotes pneumonia. The engineering objective was to improve the earlier project’s accuracy beyond 90%, while keeping an actual quantum algorithm in the inference path. The resulting architecture is a hybrid model: classical image processing produces features, a quantum feature map supplies attention similarities, and classical layers aggregate these similarities into a prediction.

An image does not enter a quantum device directly. The encoder first extracts useful visual structure. A learned projection then compresses each token into four query or key coordinates. Those coordinates determine quantum rotation angles. The overlap between two encoded states answers the attention question: how similar is this token’s query to another token’s key?

1.2 What has actually been achieved

All ten quantum classifiers—five entangled and five product-state models—exceed 90% on the 624-image official test split. The main entangled ensemble correctly classifies 572 images and misclassifies 52. All fifteen neural models and a classical linear control are retained. Their complete prediction arrays, training histories, best checkpoints, resumable checkpoints, and verification records accompany this report.

The current quantum algorithm is *compute-uncompute fidelity estimation*. Training uses a differentiable statevector simulator that evaluates the same gate circuit. Independent inference executes the corresponding circuit in PennyLane. Neither execution uses a physical quantum processing unit (QPU); there are no sampled measurement shots in this accuracy result.

1.3 What this result cannot establish

The new entangled ensemble improves the older 88.9423% reference by approximately 2.72 percentage points, equivalent to 17 more correct predictions on this cohort. However, the new experiment also adds external ImageNet pretraining and higher-resolution source images. Therefore this improvement cannot be attributed exclusively to quantum attention. The matched classical ensemble is more accurate than the quantum ensemble. A successful course project is supported; a claim that the quantum method is the best classifier or a newly invented publishable algorithm is not supported by these results.

The test cohort was inspected during earlier research. Validation-only selection within the new experiment prevents direct test-based checkpoint selection, but does not turn this repeatedly examined cohort into an independent confirmatory test. New external data and a prospectively fixed protocol would be needed for a stronger claim.

Chapter 2

Data and preprocessing

2.1 Official data and split roles

The native-resolution archive is the official MedMNIST+ PneumoniaMNIST 224-pixel dataset [1]. It contains the following fixed splits; the project does not rearrange, subset, or merge them. The class distributions differ: approximately 74.2% of training and validation images

Split	Images	Normal	Pneumonia
Training	4,708	1,214	3,494
Validation	524	135	389
Test	624	234	390

Table 2.1: Official split counts. Training supplies gradients, validation supplies selection and calibration, and test supplies evaluation.

are positive, compared with 62.5% of test images. Class weighting addresses imbalance in the training objective. This difference alone does not prove why an earlier model made errors.

The native archive is 214,384,716 bytes. Its official MD5 is

`d6a3c71de1b945ea11211b03746c1fe1`.

The smaller 28-pixel reference archive is retained for cross-resolution correspondence checks. Split labels are exactly equal between the two archives. Each native image is area-downsampled to 28 pixels and compared with the same-index reference image using centered Pearson correlation. More than 99% of each split must exceed 0.9 correlation. The measured fractions are approximately 0.999788 for training and 1.0 for validation and test. This supports correspondence of the supplied images and their ordering; it is not a patient-identity audit.

2.2 Why native resolution matters

Upscaling a 28-pixel image to 224 pixels changes its dimensions, but cannot recover anatomical detail discarded by the original downsampling. Using the supplied native 224-pixel archive provides a different input-information regime. The earlier experiment that fed enlarged 28-pixel images into the pretrained encoder reached only 86.70% quantum ensemble accuracy. The native experiment retains the same pretrained-head training recipe and changes the raw archive and experiment namespace. Both resolution regimes remain documented.

2.3 Image transformations

The data loader represents grayscale pixels in $[-1, 1]$. Before CNN inference, they are mapped back to $[0, 1]$ by $(x + 1)/2$. Grayscale is repeated across three channels to satisfy the pretrained network’s RGB input interface; repeating a channel introduces no color information.

The official `IMAGENET1K_V1` weight transform resizes the shorter side to 256 pixels and center-crops to 224×224 , followed by channel normalization [2]:

$$x'_c = \frac{x_c - \mu_c}{\sigma_c}, \quad \boldsymbol{\mu} = (0.485, 0.456, 0.406), \quad \boldsymbol{\sigma} = (0.229, 0.224, 0.225). \quad (2.1)$$

Thus “native 224” describes the source archive, while the CNN still uses its prescribed resize/crop transformation. The report does not imply that every source pixel survives the center crop.

Two feature views are cached for every training image: the original image and a horizontal flip applied before the CNN transform. Validation and test have only the canonical view. During each training batch, the head randomly selects one of the two cached views for each example. The final native experiment does not use the older branch’s random affine, brightness, or contrast augmentation.

2.4 Training-only feature normalization

The CNN yields feature arrays of shape $[V, N, 4, 512]$, where $V = 2$ for training and $V = 1$ otherwise. A mean and sample standard deviation are estimated for each of the 512 channels over training views, images, and spatial tokens:

$$\hat{f}_{n,t,c} = \frac{f_{n,t,c} - \mu_c^{\text{train}}}{\max(\sigma_c^{\text{train}}, 10^{-4})}. \quad (2.2)$$

The same stored statistics transform validation and test. No validation or test feature contributes to these fitted statistics. Caching avoids repeating the frozen CNN computation every epoch and preserves exact feature provenance. Caching a split is not fitting the model on that split.

Chapter 3

The classical image encoder and transformer

3.1 Frozen ResNet-18 transfer learning

ResNet-18 learns image representations through convolutions and residual connections. A residual block adds an input pathway to a learned transformation, so its output is conceptually $x + F(x)$. This enables the network to build representations without forcing every block to reconstruct its input from scratch.

The project loads public ImageNet-1K weights, removes the network’s original average-pooling and classification layers, and adds adaptive average pooling to a 2×2 spatial grid. At 224-pixel CNN input, the final convolutional feature map has 512 channels; pooling produces four spatial tokens with 512 features each. These tokens summarize broad spatial regions rather than raw image patches.

The encoder contains 11,176,512 parameters. All are frozen, and the encoder remains in evaluation mode: neither gradients nor batch-normalization running statistics are updated by PneuoniaMNIST. The public weight SHA256 is

f37072fd47e89c5e827621c5baffa7500819f7896bbacec160b1a16c560e07ec.

External pretraining is part of the method and must be stated whenever its results are compared with models trained without it. Hybrid transfer learning has established prior art [3]; the use of a pretrained classical encoder followed by a quantum component is not itself a new algorithm.

3.2 Token projection and sequence construction

Each normalized 512-dimensional token passes through a trainable affine projection into 32 dimensions. A trainable classification token (CLS) is prepended, and learned positional vectors distinguish the five sequence positions:

$$X_0 = \text{Dropout}([x_{\text{CLS}}; FW_P + b_P] + P), \quad X_0 \in \mathbb{R}^{B \times 5 \times 32}. \quad (3.1)$$

The CLS token accumulates information from all four spatial tokens. The head’s legacy configuration contains `image_size=28` and `patch_size=14`; these fields determine only four positional-token slots. The original patch embedding layer is explicitly deleted. They do not resize the images or imply that the new encoder uses 28-pixel input.

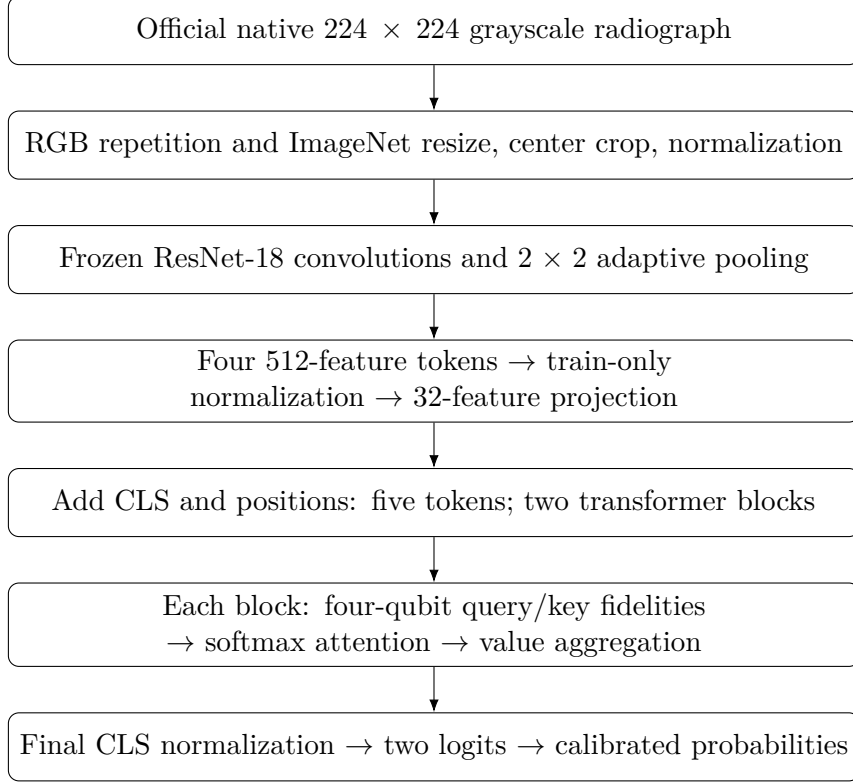


Figure 3.1: The complete hybrid inference pipeline. The quantum component supplies similarities; image encoding and value processing are classical.

3.3 Two pre-normalized transformer blocks

Each block computes

$$X' = X + \text{Attention}(\text{LayerNorm}_1(X)), \quad (3.2)$$

$$X'' = X' + \text{MLP}(\text{LayerNorm}_2(X')). \quad (3.3)$$

The MLP maps $32 \rightarrow 64 \rightarrow 32$ with GELU activation and dropout. Residual additions preserve an information pathway through the block. Layer normalization stabilizes the scale of each token’s features. Dropout probability is 0.1, including attention-weight dropout during training. Dropout is disabled in evaluation.

The final layer normalizes the CLS representation and applies a trainable linear map $32 \rightarrow 2$ to produce normal and pneumonia logits. All three neural heads have exactly 30,658 trainable parameters. The CNN parameters are not included in that trainable count.

Chapter 4

Quantum algorithms: gates, encoding, and fidelity

4.1 Qubits and the encoded state

A qubit is represented as $a|0\rangle + b|1\rangle$, where $|a|^2 + |b|^2 = 1$. Four qubits have $2^4 = 16$ computational-basis amplitudes. The feature map starts at $|0000\rangle$ and applies unitary gates determined by four real coordinates.

For each attention head, learned affine maps produce $q, k \in \mathbb{R}^4$. Each coordinate is converted into an angle using

$$\theta_i(z) = \frac{\pi}{2}(\tanh z_i + 1). \quad (4.1)$$

This bounds each angle between 0 and π . The mapping is differentiable, but large-magnitude coordinates can saturate the hyperbolic tangent and reduce its derivative. The model learns the query and key projection weights; it does not have an additional independent collection of variational gate parameters.

4.2 Rotation gates

The matrices used in this project are

$$R_Y(\theta) = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}, \quad R_Z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}. \quad (4.2)$$

An initial R_Y creates angle-dependent amplitudes:

$$R_Y(\theta)|0\rangle = \cos(\theta/2)|0\rangle + \sin(\theta/2)|1\rangle. \quad (4.3)$$

R_Z introduces a relative complex phase. A later R_Y can mix amplitudes after that phase change, allowing the final state to depend on both amplitude and phase processing.

4.3 The product-state feature map

The product control applies one $R_Y(\theta_i)$ to each wire, with no entangling gates:

$$|\psi_{\text{product}}(\boldsymbol{\theta})\rangle = \bigotimes_{i=0}^3 [\cos(\theta_i/2)|0\rangle + \sin(\theta_i/2)|1\rangle]. \quad (4.4)$$

Its fidelity has a direct formula:

$$F_{\text{product}}(q, k) = \prod_{i=0}^3 \cos^2\left(\frac{\theta_i(q) - \theta_i(k)}{2}\right). \quad (4.5)$$

This identity explains why the product kernel is efficiently classically computable. The current experiment nevertheless implements its four-qubit gates and independently checks its complete inference in PennyLane. Using a quantum-state definition is not a claim of classical computational difficulty.

For example, equal angle vectors yield fidelity 1. If three coordinates match and the remaining coordinate differs by $\pi/2$, the fidelity is $\cos^2(\pi/4) = 1/2$. If one coordinate differs by π , the states are orthogonal and fidelity is 0. These examples describe the formula rather than a trained radiograph prediction.

4.4 The entangled, data-reuploading feature map

The actual chronological gate order is:

1. Initialize four qubits in $|0000\rangle$.
2. Apply $R_Y(\theta_i)$ on wires $i = 0, 1, 2, 3$.
3. Apply $\text{CNOT}(0, 1)$, then $\text{CNOT}(1, 2)$, then $\text{CNOT}(2, 3)$.
4. On each wire, apply $R_Z(\theta_i)$ followed by $R_Y(\theta_i/2)$.

A CNOT flips its target when its control is $|1\rangle$. In the computational basis it maps $|a, b\rangle$ to $|a, b \oplus a\rangle$. On suitable superpositions it creates a joint state that cannot be factored into independent single-qubit states. The circuit can entangle its wires, although particular inputs need not produce entanglement.

With operators multiplying right to left, define

$$E(\theta) = \bigotimes_i R_Y(\theta_i), \quad (4.6)$$

$$C = \text{CNOT}_{2,3} \text{CNOT}_{1,2} \text{CNOT}_{0,1}, \quad (4.7)$$

$$D(\theta) = \bigotimes_i [R_Y(\theta_i/2)R_Z(\theta_i)], \quad (4.8)$$

$$U(\theta) = D(\theta)CE(\theta), \quad |\psi(\theta)\rangle = U(\theta)|0000\rangle. \quad (4.9)$$

This is 12 single-qubit rotations and three CNOTs, or 15 primitive gates per state preparation before any compiler optimization. Data re-uploading means the same encoded coordinates appear again after the entangling stage. Re-uploading is an established approach [4]; this specific implementation does not inherit a universal-classification theorem simply because it uses repeated encoding.

4.5 Why the second encoding stage is important

If one merely applies the same fixed entangling unitary C after both product encodings, the inner product is unchanged:

$$\langle 0|E(k)^\dagger C^\dagger CE(q)|0\rangle = \langle 0|E(k)^\dagger E(q)|0\rangle. \quad (4.10)$$

Therefore adding a common CNOT chain at the end alone would not change the fidelity kernel. In the implemented circuit, the input-dependent $D(k)^\dagger D(q)$ lies between the two entangling operators, so this simple cancellation does not apply. That gives a concrete mathematical reason for placing input-dependent rotations after the CNOT chain. It does not guarantee higher accuracy.

The product ablation removes both the CNOT chain and the second encoding stage. Consequently, differences between product and entangled models cannot isolate the contribution of entanglement alone. An isolating experiment would hold the second encoding fixed while varying only the CNOTs.

4.6 Compute–uncompute fidelity estimation

The algorithm prepares a query state and then uncomputes a key state:

$$|\phi\rangle = U(\theta(k))^\dagger U(\theta(q))|0000\rangle. \quad (4.11)$$

Its all-zero measurement probability is

$$p(0000) = \left| \langle 0000 | U(\theta(k))^\dagger U(\theta(q)) | 0000 \rangle \right|^2 = |\langle \psi(k) | \psi(q) \rangle|^2 = F(q, k). \quad (4.12)$$

The inverse circuit reverses the chronological gate order and negates rotation angles; CNOT is its own inverse. The entangled pair circuit has 30 primitive gates (24 rotations and six CNOTs) before optimization, and uses four qubits without an ancilla. The product pair circuit has eight rotations. A physical finite-shot execution would estimate this probability by the fraction of all-zero outcomes. This experiment obtains exact simulator probabilities instead.

The PennyLane backend returns the full 16-entry probability distribution and selects entry 0. It broadcasts multiple query–key settings through each QNode call. A call count therefore differs from a count of distinct circuit settings. No amplitude-estimation algorithm, Grover search, QAOA, VQE, or quantum support-vector machine is used in this classifier.

4.7 Earlier SWAP-test algorithms and their relation to this version

The original project includes standard and destructive SWAP-test routines, but they are not the inference algorithm used for the 91.67% result.

The standard SWAP test prepares two states in separate registers and an ancilla in $|0\rangle$. It applies a Hadamard to the ancilla, controlled SWAPs between corresponding register qubits, and a second Hadamard. The ancilla-zero probability is $p_0 = (1 + F)/2$, so $F = 2p_0 - 1$. For two four-qubit states this uses nine qubits, including the ancilla. Controlled SWAP synthesis can be costly.

The destructive SWAP test uses both four-qubit registers without the ancilla. Pairwise CNOT and Hadamard operations followed by computational-basis measurement implement a Bell-basis comparison. Assign -1 to a pair’s singlet outcome and $+1$ to the other Bell outcomes; the expectation of the product of these pair signs equals the register SWAP expectation, which is the fidelity for pure input states. It needs eight qubits and consumes the two prepared states.

Compute–uncompute instead requires a known reversible preparation circuit, allowing the same overlap to be obtained on four wires. These protocols target the same pure-state similarity, but their hardware requirements, noise behavior, and estimators differ. The older project’s finite-shot and noisy SWAP experiments must not be presented as measurements of this new classifier’s accuracy.

Chapter 5

How quantum similarities become attention

5.1 Queries, keys, and values

Each block has two attention heads. Given five normalized tokens of dimension 32, each head produces a query and key of dimension four and a value of dimension 16:

$$q_i = W_Q x_i + b_Q, \quad k_j = W_K x_j + b_K, \quad v_j = W_V x_j + b_V. \quad (5.1)$$

The implemented projections output both heads together and reshape their outputs. The total value dimension remains 32.

For every ordered pair of tokens (i, j) , quantum attention computes $S_{ij} = F(q_i, k_j)$. Although fidelity is symmetric in its two encoded states, the learned query and key projections are different. Therefore the token score matrix need not be symmetric, and its diagonal need not be one.

5.2 Softmax and aggregation

The scores are converted into row-normalized weights with fixed scale $\beta = 2$:

$$A_{ij} = \frac{\exp(2S_{ij})}{\sum_{r=1}^5 \exp(2S_{ir})}, \quad y_i = \sum_j A_{ij} v_j. \quad (5.2)$$

Fidelities lie between 0 and 1, but they are not already attention probabilities: their row sums are unconstrained. Softmax makes each attention row sum to one and accentuates larger similarities. The two heads' outputs are concatenated and passed through a classical output projection.

For a toy two-key row with fidelities 0.9 and 0.2, the first key's softmax weight is $1/(1 + e^{-1.4}) \approx 0.802$. Thus its value contributes more strongly to the query output. The circuit does not classify the image in isolation; it changes how spatial and CLS information are combined.

5.3 The parameter-matched dot control

The classical control uses the same encoder, tokens, projections, transformer depth, trainable parameter count, seeds, and training procedure. It replaces only the kernel score with

$$S_{ij}^{\text{dot}} = q_i^T k_j / \sqrt{4}. \quad (5.3)$$

The same factor $\beta = 2$ is applied before softmax. Dot scores are not bounded like fidelities, so the score distributions differ even when the architecture and trainable parameter count match. This is a controlled comparison of the implemented methods, not a proof of identical effective capacity.

5.4 Gradients and simulation

Training represents each state as a 16-element complex vector. Single-qubit rotations update amplitude pairs, and CNOT permutes amplitudes. Pairwise overlaps are obtained from complex inner products. PyTorch differentiates through rotations, phases, overlaps, softmax, and the classifier using ordinary reverse-mode automatic differentiation. Gradients flow back into the learned query/key projections and other classical head parameters. The frozen CNN receives no gradient.

The training algorithm does not use a hardware parameter-shift routine. Such a routine could be considered for a future hardware implementation, but its measurement cost and accuracy would need separate evaluation. A four-qubit statevector is small and efficiently simulated classically; no computational speedup is established here.

Chapter 6

Training and model selection, step by step

6.1 Fixed settings

Setting	Value
Seeds	0, 1, 2, 3, 4
Neural kernels	Entangled, product, dot
Epochs / batch size	16 / 64
Optimizer	AdamW, initial base LR 0.0008
Weight decay	0.001
LR schedule	Two-epoch warmup, cosine schedule with 0.01 floor factor
Feature MixUp	Beta distribution parameter $\alpha = 0.1$
EMA decay	0.95 per optimizer step
Gradient clipping	Global norm 1.0
Checkpoint criterion	Class-balanced validation NLL
Calibration	Positive scalar temperature fitted on validation
Ensemble	Equal mean of five calibrated probability arrays

Table 6.1: Frozen training recipe shared by all neural kernels.

No test score chooses a kernel’s best epoch, temperature, or ensemble weights. The same fixed recipe is reused from the preceding pretrained branch. This restriction makes the kernel comparison easier to interpret, even though the overall research sequence remains exploratory.

6.2 Class-weighted cross-entropy

For class counts n_c among N training cases, weights are $w_c = N/(2n_c)$. If $p_{i,c}$ is the model probability, the unmixed per-example loss is $-w_{y_i} \log p_{i,y_i}$. The implementation uses PyTorch’s weighted cross-entropy with its default mean reduction, which divides a batch’s weighted loss sum by the sum of its target weights, not simply by batch size. Normal images therefore receive more weight than the more numerous pneumonia images.

6.3 Feature MixUp

For a shuffled pairing $\pi(i)$ within a batch, sample one $\lambda \sim \text{Beta}(0.1, 0.1)$ and compute

$$\tilde{x}_i = \lambda x_i + (1 - \lambda) x_{\pi(i)}, \quad (6.1)$$

$$L = \lambda \text{CE}_w(z, y) + (1 - \lambda) \text{CE}_w(z, y_\pi). \quad (6.2)$$

MixUp acts on normalized CNN tokens rather than on raw images or quantum amplitudes. The resulting mixed tokens pass through the same learned projections and circuit encoding. This encourages smoother predictions between feature examples. With $\alpha = 0.1$, many draws remain close to one of the two examples; it is not uniform interpolation.

6.4 AdamW and the learning-rate schedule

AdamW maintains first and second moments of gradients, approximately

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (6.3)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (6.4)$$

$$\vartheta_{t+1} = (1 - \eta_t \lambda_w) \vartheta_t - \eta_t \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon). \quad (6.5)$$

Bias correction gives $\hat{m}_t, \hat{v}_t$. The implementation uses PyTorch defaults for the Adam moment factors and epsilon. Decoupled weight decay reduces parameter magnitude independently of the adaptive gradient scaling. These are classical optimizer operations.

The scheduler multiplies the base LR by the following function of its zero-based scheduler index e , with $w = 2$ and $E = 16$:

$$s(e) = \begin{cases} (e + 1)/w, & e < w, \\ 0.01 + 0.99 [1 + \cos(\pi(e - w)/(E - w))]/2, & e \geq w. \end{cases} \quad (6.6)$$

Warmup reduces the early step size; the later cosine schedule decreases it smoothly. The scheduler advances after each completed epoch. Gradients are clipped to global norm 1 before the optimizer update.

6.5 Exponential moving average and checkpoint selection

After each optimizer step, a second parameter copy is updated as

$$\vartheta_{\text{EMA}} \leftarrow 0.95 \vartheta_{\text{EMA}} + 0.05 \vartheta. \quad (6.7)$$

EMA smooths recent training updates. It starts from the initialized model and has no separate optimizer. At every epoch, the EMA model is evaluated without dropout on canonical validation features. The selection criterion gives equal weight to the two classes:

$$L_{\text{balanced}} = \frac{1}{2} \sum_{c=0}^1 \frac{1}{N_c^{\text{val}}} \sum_{i: y_i=c} -\log p_{i,c}. \quad (6.8)$$

The best EMA state is saved whenever this score improves. The final evaluation loads this saved EMA state, rather than automatically using the last epoch or the last raw training parameters.

6.6 Temperature calibration and ensemble construction

A positive scalar T rescales logits before softmax:

$$p_{i,c}(T) = \frac{\exp(z_{i,c}/T)}{\sum_r \exp(z_{i,r}/T)}. \quad (6.9)$$

The implemented scalar search minimizes ordinary validation log loss over $\log T \in [-3, 3]$. This differs from the class-balanced checkpoint criterion. A positive temperature preserves each individual model’s argmax, while altering confidence and potentially an average-probability ensemble’s decisions.

For each kernel, five calibrated probability arrays are averaged with equal weights:

$$\bar{p}_i = \frac{1}{5} \sum_{s=0}^4 p_i^{(s)}(T_s), \quad \hat{y}_i = \arg \max_c \bar{p}_{i,c}. \quad (6.10)$$

This ensemble is one prediction set on the 624 images. It is not five independent test repetitions. There is no test-optimized threshold, known-test-prevalence constraint, or target-prior EM adaptation in the new native experiment.

6.7 Resumability and reproducibility

The resumable checkpoint retains current model parameters, EMA parameters, optimizer moments, scheduler state, epoch, best validation criterion, elapsed time, and random states for Python, NumPy, PyTorch, and the data-loader generator. Restoring only the model would not reproduce future batches, MixUp draws, view choices, and dropout masks. Tests verify bit-exact interrupted versus uninterrupted training in a short stochastic fixture.

6.8 Training algorithm

Listing 6.1: Operational training sequence; the exact Python implementation is included in the appendix.

```

Verify manifest, source, weight, and feature-cache hashes.
For each kernel in {entangled, product, dot}:
  For seed in {0,1,2,3,4}:
    Initialize the matched head, EMA copy, AdamW, and scheduler.
    Restore all states if a resumable checkpoint exists.
    Repeat for 16 epochs:
      Shuffle all training indices.
      For each batch:
        Randomly select original or flipped cached features.
        Mix feature examples using a Beta(0.1,0.1) draw.
        Predict using quantum fidelity or matched dot attention.
        Backpropagate mixed class-weighted cross-entropy.
        Clip gradients; update AdamW; update EMA.
      Evaluate EMA on canonical validation features.
      Save EMA if class-balanced validation NLL improves.
      Advance scheduler; save resumable state.
    Reload best EMA; fit validation temperature.
    Save complete validation and test logits/probabilities.
For each kernel, average its five calibrated probability arrays.
Independently reproduce every quantum test overlap in PennyLane.
```

Chapter 7

Controls, evaluation, and measured results

7.1 Classical linear control

The linear control averages the four spatial features to 512 coordinates and fits class-balanced logistic regression. It uses both training views as training observations; validation and test use only their canonical views. It has 512 coefficients and one intercept. Regularization candidates $C \in \{0.01, 0.1, 1, 10\}$ are scored by class-balanced validation NLL, selecting $C = 0.1$. Its validation temperature is fitted by the same ordinary-log-loss calibration rule. This control asks how much useful discrimination is already present in the frozen CNN features without transformer attention.

7.2 Metrics and their interpretation

Accuracy is $(TP+TN)/N$. Sensitivity is $TP/(TP+FN)$ for pneumonia, specificity is $TN/(TN+FP)$ for normal images, and balanced accuracy is their mean. Precision is $TP/(TP+FP)$, and F_1 is the harmonic mean of precision and sensitivity. AUROC evaluates ordering of positive versus negative examples across thresholds; it does not measure calibration.

Negative log likelihood penalizes confident mistakes. Binary Brier score averages $(p_i - y_i)^2$. Expected calibration error groups maximum predicted confidence into ten bins and weights absolute confidence–accuracy gaps by each bin’s sample fraction. Each metric answers a different question. High overall accuracy can coexist with lower normal-class specificity, so the complete metrics and confusion matrices must remain available.

7.3 Main accuracy and AUROC results

Model	Mean accuracy $\pm$ SD	Mean AUROC $\pm$ SD	Ensemble accuracy
Entangled quantum	91.60 $\pm$ 0.58%	0.98394 $\pm$ 0.00394	91.67%
Product quantum	91.22 $\pm$ 0.29%	0.98443 $\pm$ 0.00142	91.19%
Matched dot	91.15 $\pm$ 0.13%	0.98336 $\pm$ 0.00226	92.15%
Linear classifier	91.03%	0.98581	Not an ensemble

Table 7.1: Five-seed neural results and one deterministic linear control. SD describes variation over training seeds on the same test cohort.

The entangled ensemble has AUROC 0.98585 and 572 correct cases. The product ensemble has AUROC 0.98597 and 569 correct cases. The dot ensemble has 575 correct cases. The linear

model has 568 correct cases. The three-case advantage of the dot ensemble over entangled attention is descriptive; it is not automatically a statistically established advantage.

Seed	Entangled accuracy	Product accuracy	Dot accuracy
0	90.865%	91.186%	91.026%
1	91.987%	91.506%	91.186%
2	91.667%	90.865%	91.026%
3	91.186%	91.506%	91.346%
4	92.308%	91.026%	91.186%

Table 7.2: Every quantum seed exceeds 90%; no failed seed is removed. Rounded values are derived from the retained per-seed CSV.

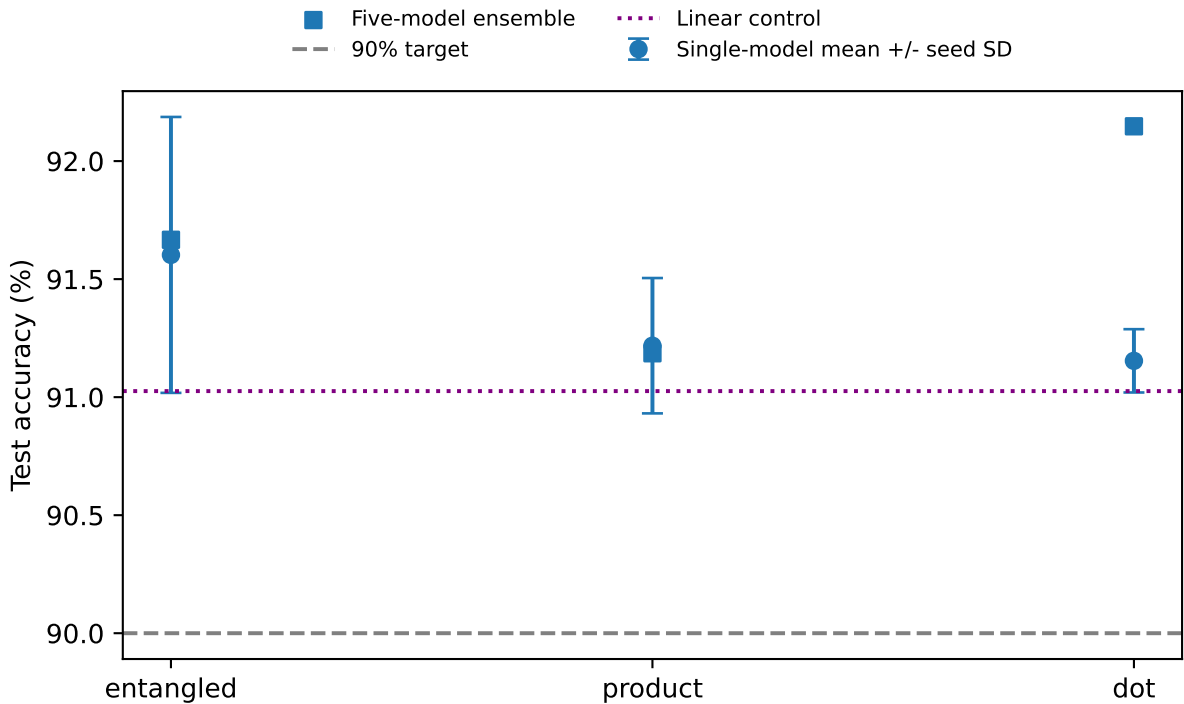


Figure 7.1: Retained comparison figure: single-model mean and standard deviation, ensemble outcomes, and classical linear control.

7.4 Paired seed statistics

Seeds are paired across kernels. Entangled minus product mean accuracy is +0.38462 percentage points, and entangled minus dot is +0.44872 points. These single-model means do not imply that their ensembles follow the same ordering.

The retained analysis performs exact sign-flip comparisons of paired seed differences, bootstraps seed means, and applies Holm correction to four comparisons: accuracy and AUROC against each of two controls. All four Holm-adjusted p values are 1.0. With only five seeds, the smallest nonzero two-sided exact sign-flip p is 0.0625. The bootstrap intervals characterize seed variation on this fixed cohort, not generalization to new patients.

Contrast	Metric	Mean effect	Bootstrap 95% interval
Entangled – product	Accuracy, points	+0.38462	[−0.16026, +0.92949]
Entangled – dot	Accuracy, points	+0.44872	[0.00000, +0.89744]
Entangled – product	AUROC	−0.00049	[−0.00353, +0.00192]
Entangled – dot	AUROC	+0.00058	[−0.00185, +0.00333]

Table 7.3: Exploratory paired-seed effects. All associated Holm-adjusted p values equal 1.0.

7.5 Paired ensemble comparisons

McNemar’s test focuses on images for which two classifiers disagree in correctness. The entangled–dot ensemble comparison has discordant counts 7 and 4 and exact two-sided $p = 0.548828$. Entangled–product has discordant counts 2 and 5 and $p = 0.453125$. Entangled–linear has counts 5 and 9 and $p = 0.42395$. The direction of each count should be read from the saved analysis rather than inferred from the count order. None establishes a quantum accuracy benefit.

The analysis also retains paired DeLong comparisons of AUROC. Such paired-example statistics use the shared labeled examples, whereas seed comparisons use training-seed differences. These are different uncertainty sources. Repeated inspection of the same test cohort limits confirmatory interpretation of both.

Chapter 8

Independent verification of the quantum path

8.1 What is checked

The analyzer verifies the frozen source and configuration hashes, feature metadata, encoder weights, dataset checksum, sample IDs, and labels. It recomputes normalization from the original training feature bank. It then recomputes CNN features from every original validation and test image and compares them with the stored banks.

For all fifteen neural checkpoints, inference is rerun and saved logits, probabilities, and reported metrics are checked. The linear model’s coefficient array is also used to reproduce its output. This establishes that the result files correspond to the retained models rather than merely being typed into a table.

8.2 Complete PennyLane inference

Each of the ten quantum checkpoints replaces its training statevector-overlap backend with an independent PennyLane `default.qubit` compute–uncompute circuit. It evaluates every ordered token pair in each head and block for every test image. With $L = 2$ blocks, $H = 2$ heads, and $N = 5$ tokens, there are

$$LHN^2 = 2 \cdot 2 \cdot 25 = 100 \quad (8.1)$$

circuit settings per image. This includes self-position pairs; query and key projections differ, so those entries cannot generally be assumed to have fidelity one.

Across 624 images and ten classifiers, the independently executed total is

$$624 \cdot 10 \cdot 100 = 624,000 \quad (8.2)$$

four-qubit settings, batched into 780 actual QNode calls. The verification records zero sampled shots because the simulator returns ideal probabilities. Zero shots means analytic simulator execution, not a probability estimate obtained without resource cost on hardware.

All argmax predictions match between the PyTorch gate implementation and PennyLane. The maximum observed logit disagreement is below 10^{-6} , consistent with their differing numerical precision. The independently evaluated quantum ensembles reproduce the reported 91.67% and 91.19% accuracies.

8.3 Tests and provenance

The recorded complete suite has 24 passing tests. Relevant checks compare gate-prepared states with PennyLane, compare probabilities, perform numerical gradient checks, verify that suit-

able entangled outputs are nonseparable, check exact parameter matching, verify whole-model backend agreement, enforce training-only normalization, and check stochastic training resumption. Other tests cover earlier project components and remain included to preserve dependency coverage.

This does not prove behavior under decoherence, gate errors, readout errors, finite-shot attention noise, hardware connectivity constraints, or an independent clinical population. Those conditions were not part of the new accuracy execution.

Chapter 9

Earlier attempts and why they remain relevant

9.1 The older 88.94% reference

The earlier small-model ensemble with target-prior adjustment reached 88.9423% for both fidelity and dot attention. Its target adaptation used unlabeled prediction information; labels were reserved for scoring. Earlier illustrative calculations that optimized a threshold using test labels or supplied the known positive count are not deployable methods and are excluded from the valid accuracy claim.

The new model does not use that target adaptation. It uses canonical test images, validation-calibrated probabilities, equal ensemble weights, and argmax. Consequently, the 2.72-point increase is a comparison of different complete pipelines, not an isolated optimizer or quantum-kernel ablation.

9.2 Other unsuccessful branches

Branch	Relevant ensemble accuracy
Original small ensemble with valid prior adjustment	88.94%
Robust small model, entangled quantum attention	89.10%
Robust small model, product quantum attention	89.74%
Pretrained ResNet with enlarged 28-pixel source, entangled	86.70%
Pretrained ResNet with enlarged 28-pixel source, product	86.70%
Native-224 pretrained encoder, entangled	91.67%
Native-224 pretrained encoder, product	91.19%

Table 9.1: Selected earlier outcomes and the final resolution branch. These are separate regimes rather than a single controlled treatment comparison.

The robust small-backbone study piloted AdamW and sharpness-aware minimization (SAM). SAM perturbs parameters in the gradient direction, evaluates a second loss at the perturbed parameters, then updates the restored parameters. The shared pilot’s average class-balanced validation NLL favored AdamW (approximately 0.19343 versus 0.22820 for SAM). SAM was not used in the final native branch. A longer-training branch and a trained convolutional-token branch also failed to establish a meaningful quantum advantage.

The separate bias-aware shot-allocation project studied measurement-error correction and sampling policy, rather than this ideal native classifier. Its joint correction/allocation primary MSE was approximately 55.1% worse than the combined raw reference. That negative result remains documented. Neither its finite-shot measurements nor its accuracy can be substituted for measurements of the updated pretrained architecture.

9.3 What is plausible, and what is measured

Higher-resolution images plausibly expose useful detail and pretrained CNN representations plausibly reduce the burden on a small downstream head. The native and enlarged-28 outcomes are consistent with this explanation. However, the current results do not decompose the gain into separately randomized contributions from resolution, pretraining, optimizer, MixUp, EMA, and attention. A factorial or carefully planned ablation study would be needed for causal attribution.

Chapter 10

Reproduction and the UPDATED package

10.1 Directory map

Path	Purpose
<code>research/</code>	Byte-preserved source and supporting modules, including earlier algorithms required by imports and tests.
<code>configs/</code>	Original manifests and frozen hash locks; native manifest defines the completed study.
<code>runs/native_quantum_accuracy/</code>	Fifteen trained heads, linear coefficients, configurations, histories, predictions, and resumable checkpoints.
<code>artifacts/native_quantum_features/</code>	Frozen raw feature banks, training-only normalization, provenance metadata, and alignment audit.
<code>artifacts/native_quantum_accuracy/</code>	Summary, per-seed CSV, full verification, independent circuit predictions, ensembles, and plots.
<code>pretrained_models/</code>	Verified public ResNet-18 ImageNet weights.
<code>data/</code>	Small reference archive and local native-224 archive.
<code>tests/, verification/</code>	Executable tests, recorded logs, claims and known-failure documentation.
<code>environment/</code>	Recorded environment and full dependency lock.
<code>fetch_assets.py</code>	Restore the public raw archive after cloning and verify the encoder weights.
<code>verify_package.py</code>	Check packaged file bytes against the SHA256 inventory.
<code>PACKAGE_CONTENTS.json</code>	Relative paths, file sizes, and exact SHA256 records.

10.2 Running the retained study

All commands assume the current directory is `UPDATED`, because the original experiment uses relative paths. The recorded environment was Python 3.14.2, PyTorch 2.14 CPU, torchvision 0.29, and PennyLane 0.45.1, with no available GPU or QPU. The environment lock records the actual installed packages; a new clean installation has not independently been validated merely

by writing this report.

Listing 10.1: Reproduce from the standalone folder.

```
cd UPDATED
python -m pip install -r requirements.txt
python fetch_assets.py
python verify_package.py
python tests/run_suite.py
python -m research.quantum_native_accuracy analyze
```

The analysis recomputes every validation/test feature and every quantum test overlap, so it is more expensive than loading a saved summary. On a CPU it can take several minutes. The native adapter automatically requests full circuit verification.

10.3 Preparing or training a new study

The executable exposes **prepare**, **freeze**, **run**, and **analyze**. Prepare builds feature banks; freeze records source, manifest, weight, and feature hashes; run trains or resumes conditions; analyze verifies and reports them. The completed package is already prepared, frozen, and trained. Running **run** reuses completed results instead of silently retraining them.

To conduct a fresh experiment, create a separate copy with a distinct experiment namespace and new provenance. Do not edit the frozen computational files in this package or refreeze them after looking at these test outcomes. Regenerating caches with a different software stack may produce numerically close but byte-different files; such changes do not satisfy the old cache hash lock and should form a new experiment rather than replacing historical evidence.

10.4 GitHub distribution and asset handling

The local package includes the official 214 MB native archive. It exceeds the normal GitHub per-file limit, so that archive is ignored in ordinary Git and restored through the included fixed-URL, checksum-verified downloader. Its size and SHA256 are still recorded in the package inventory. The smaller reference archive, encoder weights, feature caches, checkpoints, predictions, and report are distributed with the source.

The copying process preserves original source bytes so the existing freeze remains meaningful after relocation. The earlier root documents, dashboard, presentation, and experiments are not overwritten. A copy of the legacy dashboard and one original small-model checkpoint is included only for its existing integration tests; it does not serve the new native classifier. This package's report describes the updated architecture; the original presentation remains a record of the earlier study.

Chapter 11

Contribution, limitations, and defensible presentation

11.1 A precise contribution statement

The project implements and verifies a complete hybrid image classifier in which a frozen pre-trained encoder feeds a compact transformer using four-qubit entangled state-fidelity attention. It evaluates that classifier alongside a product-state circuit, a parameter-matched dot-attention head, and a simple linear classifier under a shared protocol. It retains failed attempts and independently checks complete circuit inference. These are concrete engineering and experimental contributions.

A defensible presentation statement is:

We built a hybrid classifier using pretrained image features and four-qubit quantum fidelity attention. Its entangled five-model ensemble achieved 91.67% on all 624 official test images, and independent PennyLane inference reproduced those decisions. The gain over our earlier model accompanies higher-resolution data and external pretraining; our matched classical ensemble reached 92.15%, so we do not claim quantum superiority.

11.2 Publication and novelty

Transfer learning, angle encoding, repeated data encoding, entangling circuits, and fidelity-based comparison have prior art. Their combination with an accuracy number does not automatically constitute a new publishable algorithm. This report does not claim the four-qubit circuit is an invention absent from the literature. The retained novelty-gate documents record the prior-art checks performed before the new implementation.

Possible stronger research directions include a genuinely isolating entanglement ablation, prospectively fixed testing on another cohort, calibrated finite-shot resource comparisons, or a method addressing a demonstrated limitation with a clearly distinguished mathematical contribution. These are future possibilities, not completed results. More tuning on the already inspected test split would not provide independent evidence.

11.3 Remaining limitations

- Results use an already inspected official cohort, not independent external confirmation.
- Higher resolution and ImageNet pretraining confound attribution of improvement over the old regime.

- The best matched classical ensemble is more accurate; no statistically established quantum advantage appears.
- Product versus entangled removes two circuit components, preventing attribution solely to entanglement.
- Four-qubit ideal simulation says nothing by itself about noisy or finite-shot hardware accuracy.
- Five seeds give limited statistical resolution. Seeds are not independent patient populations.
- Dataset ordering checks do not prove absence of patient overlap or clinical deployment validity.
- The native classifier’s measured quantum resources exclude hardware execution, transpilation, and shots; frozen CNN costs also remain classical costs.

Appendix A

Implementation source used for the result

The following files are the retained implementation, not illustrative pseudocode. Auxiliary modules and exact configurations are supplied in the package. The robust module also contains the older augmentation and SAM utilities; those routines are not activated by the native transfer trainer. The native adapter changes runtime namespaces and loading resolution without changing the preceding frozen computational files.

A.1 Quantum gates and attention

```
"""Differentiable four-qubit circuit attention and generalization utilities.

Torch executes gates on small statevectors for training. PennyLane can execute
the matching compute-uncompute circuit for every inference overlap. Both are
classical quantum-circuit simulators; neither implies physical QPU execution.
"""
import math
from functools import lru_cache
import numpy as np
import torch
from torch import nn
from torch.nn import functional as F
from .models import Attention, MatchedViT, angles

def rotation(state, theta, wire, d, kind):
    shape=state.shape
    blocks=state.reshape(*shape[:-1], 2**wire, 2, 2**(d-wire-1))
    a,b=blocks.select(-2,0), blocks.select(-2,1)
    theta=theta[..., None, None]
    if kind=='y':
        c,s=torch.cos(theta/2), torch.sin(theta/2)
        out=torch.stack([c*a-s*b, s*a+c*b], dim=-2)
    else:
        out=torch.stack([torch.exp(-.5j*theta)*a, torch.exp(.5j*theta)*b], dim=-2)
    return out.reshape(shape)

@lru_cache(maxsize=32)
def cnot_permutation(d, control, target):
    i=torch.arange(2**d)
    return i^(((i>>(d-control-1))&1)<<(d-target-1))

def feature_states(theta, entangled=True):
    d=theta.shape[-1]
    if d!=4:
        raise ValueError('This follow-up uses exactly four qubits')
    dtype=torch.complex128 if theta.dtype==torch.float64 else torch.complex64
```

```

state=torch.zeros((*theta.shape[:-1],2**d),device=theta.device,dtype=dtype)
state[...,0]=1
for wire in range(d):
    state=rotation(state,theta[...,wire],wire,d,'y')
if entangled:
    for wire in range(d-1):
        state=state.index_select(-1,cnot_permutation(d,wire,wire+1).to(theta.device))
    for wire in range(d):
        state=rotation(state,theta[...,wire],wire,d,'z')
        state=rotation(state,.5*theta[...,wire],wire,d,'y')
return state

def feature_circuit(theta,entangled=True):
    import pennylane as qml
    for wire in range(4):
        qml.RY(theta[...,wire],wire)
    if entangled:
        for wire in range(3):
            qml.CNOT([wire,wire+1])
        for wire in range(4):
            qml.RZ(theta[...,wire],wire)
            qml.RY(.5*theta[...,wire],wire)

class PennyLaneOverlap:
    def __init__(self,entangled=True,chunk_size=4096):
        import pennylane as qml
        self.entangled=entangled
        self.chunk_size=chunk_size
        self.qnode_calls=0
        self.circuit_settings=0
        self.shots=0
        device=qml.device('default.qubit',wires=4)
        @qml.qnode(device)
        def overlap(q,k):
            feature_circuit(q,self.entangled)
            qml.adjoint(feature_circuit)(k,self.entangled)
            return qml.probs(wires=range(4))
        self.circuit=overlap

    def __call__(self,qa,ka):
        qa,ka=np.broadcast_arrays(np.asarray(qa,dtype=np.float64),np.asarray(ka,dtype=np.float64))
        shape=qa.shape[:-1]
        qa,ka=qa.reshape(-1,4),ka.reshape(-1,4)
        result=[]
        for start in range(0,len(qa),self.chunk_size):
            p=np.asarray(self.circuit(qa[start:start+self.chunk_size],ka[start:start+self.chunk_size]))
            result.append(p[:,0])
            self.qnode_calls+=1
            self.circuit_settings+=len(p)
        return np.concatenate(result).reshape(shape)

class CircuitAttention(Attention):
    def __init__(self,c,kernel):
        super().__init__(c)
        self.kernel=kernel
        self.circuit_backend=None
        self.prepared_states=0

    def forward(self,x):
        b,n,e=x.shape
        q=self.q(x).view(b,n,self.heads,self.dk).transpose(1,2)
        k=self.k(x).view(b,n,self.heads,self.dk).transpose(1,2)
        v=self.v(x).view(b,n,self.heads,self.dv).transpose(1,2)
        if self.kernel=='dot':
            scores=(q@k.transpose(-1,-2))/math.sqrt(self.dk)
        else:
            qa,ka=angles(q),angles(k)
            if self.circuit_backend is None:
                qs=feature_states(qa,self.kernel=='entangled')
                ks=feature_states(ka,self.kernel=='entangled')

```

```

        self.prepared_states+=2*b*self.heads*n
        scores=(qs.conj()@ks.transpose(-1,-2)).abs().square()
    else:
        if self.training:
            raise ValueError('PennyLane override is inference-only')
        f=self.circuit_backend(qa.detach().numpy()[::,::,None,:],ka.detach().numpy()[::,::,None,:,:])
        scores=torch.as_tensor(f, dtype=x.dtype, device=x.device)
        attention=(self.beta*scores).softmax(-1)
        output=(self.drop(attention@v).transpose(1,2).reshape(b,n,e)
        return self.out(output)

class QuantumRobustViT(MatchedViT):
    def __init__(self,c,kernel):
        super().__init__(c)
        for block in self.blocks:
            block.attn=CircuitAttention(c,kernel)
        self.kernel=kernel

    def set_circuit_backend(self,backend):
        if self.kernel=='dot':
            raise ValueError('Dot is the classical matched control')
        for block in self.blocks:
            block.attn.circuit_backend=backend

def augment(x,enabled=True):
    if not enabled:
        return x
    b=len(x)
    # Per-image affine transforms with border padding (no cyclic wraparound).
    angle=(torch.rand(b,device=x.device)*2-1)*(10*math.pi/180)
    scale=.9+.2*torch.rand(b,device=x.device)
    shift=(torch.rand(b,2,device=x.device)*2-1)*.1
    affine=torch.zeros(b,2,3,device=x.device,dtype=x.dtype)
    affine[:,0,0]=torch.cos(angle)*scale
    affine[:,1,1]=torch.cos(angle)*scale
    affine[:,0,1]=-torch.sin(angle)*scale
    affine[:,1,0]=torch.sin(angle)*scale
    affine[:,2,2]=shift
    grid=F.affine_grid(affine,x.shape,align_corners=False)
    transformed=F.grid_sample(x,grid,mode='bilinear',padding_mode='border',align_corners=False)
    flip=torch.rand(b,device=x.device)<.5
    transformed=torch.where(flip[:,None,None],transformed.flip(-1),transformed)
    # Independent brightness/contrast changes act on [0,1] intensity.
    z=(transformed+1)/2
    contrast=.8+.4*torch.rand(b,1,1,1,device=x.device)
    brightness=(torch.rand(b,1,1,1,device=x.device)*2-1)*.1
    z=((z-.5)*contrast+.5+brightness).clamp(0,1)
    return z*2-1

class SAMStep:
    """Two-pass SAM around a base optimizer; restores parameters before updating."""
    def __init__(self,optimizer,rho=.05):
        self.optimizer=optimizer
        self.rho=rho

    def step(self,closure):
        self.optimizer.zero_grad(set_to_none=True)
        rng=torch.get_rng_state()
        loss=closure();loss.backward()
        params=[p for group in self.optimizer.param_groups for p in group['params'] if p.grad is not None]
        norm=torch.linalg.vector_norm(torch.stack([p.grad.norm() for p in params]))
        backups=[]
        with torch.no_grad():
            for p in params:
                backups.append((p,p.detach().clone()))
                p.add_(p.grad,alpha=float(self.rho/(norm+1e-12)))
        try:
            self.optimizer.zero_grad(set_to_none=True)
            torch.set_rng_state(rng) # same dropout realization in both passes
            perturbed=closure();perturbed.backward()

```

```

    finally:
        with torch.no_grad():
            for p,old in backups:
                p.copy_(old)
nn.utils.clip_grad_norm_(params,1.)
self.optimizer.step()
return loss.detach()

```

A.2 Pretrained encoder and transfer head

```

"""Frozen ImageNet image encoder feeding matched quantum/classical attention."""
import json
from pathlib import Path
import time
import numpy as np
import torch
from torch import nn
from torchvision.models import resnet18, ResNet18_Weights
from .config import Config
from .data import load_splits
from .quantum_robust import QuantumRobustViT
from .quantum_robust_experiment import digest
from .runtime import json_write, torch_save_atomic, seed_all

MANIFEST=Path('configs/pretrained_quantum_accuracy.json')
CACHE=Path('artifacts/pretrained_quantum_features')

def head_config(spec,seed,kernel):
    return Config(**(spec['head']|{'dataset':spec['dataset'],'seed':seed,
                                   'model':'dot' if kernel=='dot' else 'fidelity'}))

class QuantumTransferViT(QuantumRobustViT):
    def __init__(self,c,kernel,feature_dim=512):
        super().__init__(c,kernel)
        del self.patch
        self.feature_projection=nn.Linear(feature_dim,c.embed_dim)

    def forward(self,x):
        x=self.feature_projection(x)
        x=self.drop(torch.cat([self.cls.expand(len(x),-1,-1),x],1)+self.position)
        for block in self.blocks:
            x=block(x)
        return self.head(self.norm(x[:,0]))

def make_encoder(spec):
    path=Path(spec['backbone']['weight_file'])
    assert digest(path).startswith('f37072fd'),'Unexpected pretrained weight hash'
    model=resnet18(weights=None)
    model.load_state_dict(torch.load(path,weights_only=True))
    encoder=nn.Sequential(*list(model.children())[:-2],nn.AdaptiveAvgPool2d(spec['backbone']['pool_size']))
    encoder.requires_grad_(False).eval()
    return encoder,ResNet18_Weights.IMAGENET1K_V1.transforms()

@torch.no_grad()
def image_features(encoder,transform,images,batch_size=32,flip=False):
    pieces=[]
    for start in range(0,len(images),batch_size):
        x=(images[start:start+batch_size]+1)/2
        if flip:
            x=x.flip(-1)
        x=transform(x.expand(-1,3,-1,-1))
        pieces.append(encoder(x).flatten(2).transpose(1,2).contiguous())
    return torch.cat(pieces)

def training_stats(train):

```

```

# Only training features enter normalization; there is no target argument.
return train.mean((0,1,2)),train.std((0,1,2)).clamp_min(1e-4)

def feature_hashes():
    return {p.name:digest(p) for p in sorted(CACHE.glob('*.pt'))}

def prepare():
    spec=json.loads(MANIFEST.read_text())
    seed_all(611,spec['backbone']['threads'])
    ds,_,_,data=load_splits(head_config(spec,0,'entangled'))
    encoder,transform=make_encoder(spec)
    CACHE.mkdir(parents=True,exist_ok=True)
    if (CACHE/'metadata.json').exists():
        meta=json.loads((CACHE/'metadata.json').read_text())
        assert meta['manifest_sha256']==digest(MANIFEST)
        assert meta['weights_sha256']==digest(spec['backbone']['weight_file'])
        assert meta['dataset_sha256']==data['sha256'] and meta['files_sha256']==feature_hashes()
        print('Reused complete feature cache',flush=True)
        return
    timings={};shapes={}
    for split in ['train','val','test']:
        start=time.perf_counter()
        images,labels=ds[split].tensors
        path=CACHE/(split+'.pt')
        if path.exists():
            bank=torch.load(path,weights_only=True)
            assert torch.equal(bank['labels'],labels)
        else:
            views=[]
            for flip in ([False,True] if split=='train' else [False]):
                before=time.perf_counter()
                views.append(image_features(encoder,transform,images,spec['backbone']['feature_batch_size'],flip))
            print('Extracted',split,'flip',flip,'images',len(images),'seconds',round(time.perf_counter()-before,1),flush=True)
            bank={'features':torch.stack(views),'labels':labels,'sample_ids':torch.arange(len(labels))}
            torch_save_atomic(bank,path)
            shapes[split]=list(bank['features'].shape)
            timings[split]=time.perf_counter()-start
    train=torch.load(CACHE/'train.pt',weights_only=True)['features']
    mean,std=training_stats(train)
    torch_save_atomic({'mean':mean,'std':std},CACHE/'normalization.pt')
    json_write(CACHE/'metadata.json',{'manifest_sha256':digest(MANIFEST),
        'weights_sha256':digest(spec['backbone']['weight_file']),'dataset_sha256':data['sha256'],
        'feature_source_sha256':digest(__file__),'files_sha256':feature_hashes(),
        'shapes':shapes,'seconds':timings,'preprocessing':str(transform),
        'frozen_encoder_parameters':sum(p.numel() for p in encoder.parameters()),
        'pretraining':'ImageNet-1K','normalization_fit':'training_views_only',
        'physical_hardware':False})
    print('Complete feature cache',shapes,flush=True)

def load_features():
    stats=torch.load(CACHE/'normalization.pt',weights_only=True)
    banks={}
    for split in ['train','val','test']:
        bank=torch.load(CACHE/(split+'.pt'),weights_only=True)
        bank['features']=(bank['features']-stats['mean'])/stats['std']
        banks[split]=bank
    return banks

if __name__=='__main__':
    prepare()

```

A.3 Training, selection, and linear control

```

"""Fixed, matched pretrained-feature quantum accuracy follow-up."""

```

```

import argparse
from copy import deepcopy
from datetime import datetime, timezone
import json
import math
from pathlib import Path
import random
import time
import numpy as np
import torch
from torch.utils.data import DataLoader, TensorDataset
from sklearn.linear_model import LogisticRegression
from threadpoolctl import threadpool_limits
from .quantum_transfer import (MANIFEST, CACHE, QuantumTransferViT, head_config,
                               load_features, feature_hashes)
from .quantum_robust_experiment import digest, balanced_nll, hashes as robust_hashes
from .metrics import probabilities, classification_metrics, fit_temperature
from .runtime import seed_all, json_write, event, snapshot, torch_save_atomic
from .train import predict

ROOT=Path('runs/pretrained_quantum_accuracy')

def source_hashes():
    return robust_hashes()|{p:digest(p) for p in ['research/quantum_transfer.py',
                                                  'research/quantum_transfer_experiment.py']}

def verify_lock(spec):
    lock=json.loads(MANIFEST.with_suffix('.frozen.json').read_text())
    meta=json.loads((CACHE/'metadata.json').read_text())
    assert lock['manifest_sha256']==digest(MANIFEST) and lock['source_sha256']==source_hashes()
    assert lock['feature_metadata_sha256']==digest(CACHE/'metadata.json')
    assert lock['feature_files_sha256']==meta['files_sha256']==feature_hashes()
    assert lock['weights_sha256']==digest(spec['backbone']['weight_file'])
    return lock

def save_predictions(folder, stem, model, features, labels, temperature, details):
    z, y, seconds=predict(model, TensorDataset(features, labels), 64)
    p=probabilities(z, temperature)
    np.savez_compressed(folder/(stem+'_predictions.npz'), logits=z, probabilities=p,
                       labels=y, sample_ids=np.arange(len(y)))
    metrics=classification_metrics(y, p)
    metrics.update(details, temperature=temperature, balanced_nll=balanced_nll(z, y),
                  inference_seconds_head_only=seconds)
    json_write(folder/(stem+'_metrics.json'), metrics)
    return metrics

def run_one(spec, banks, kernel, seed):
    folder=ROOT/f'{kernel}-s{seed}'
    if (folder/'test_metrics.json').exists():
        return json.loads((folder/'test_metrics.json').read_text())
    folder.mkdir(parents=True, exist_ok=True)
    tr=spec['training']; seed_all(seed, tr['threads'])
    model=QuantumTransferViT(head_config(spec, seed, kernel), kernel)
    ema=deepcopy(model).eval()
    opt=torch.optim.AdamW(model.parameters(), lr=tr['lr'], weight_decay=tr['weight_decay'])
    def schedule(epoch):
        w=tr['warmup_epochs']
        return (epoch+1)/w if epoch<w else .01+.99*.5*(1+math.cos(math.pi*(epoch-w)/(tr['epochs']-w)))
    sched=torch.optim.lr_scheduler.LambdaLR(opt, schedule)
    bank=banks['train']; n=len(bank['labels'])
    generator=torch.Generator().manual_seed(seed+731000)
    loader=DataLoader(TensorDataset(torch.arange(n), bank['labels']), batch_size=tr['batch_size'],
                          shuffle=True, generator=generator)
    counts=torch.bincount(bank['labels'], minlength=2).float()
    weight=counts.sum()/(2*counts) if tr['class_weight'] else None
    loss_fn=torch.nn.CrossEntropyLoss(weight=weight)
    start_epoch, best, elapsed=0, float('inf'), 0.
    if not (folder/'config.json').exists():
        json_write(folder/'config.json', {'seed':seed, 'kernel':kernel, 'specification':spec})

```

```

    snapshot(folder/'initial_source')
if (folder/'last.pt').exists():
    state=torch.load(folder/'last.pt',weights_only=False)
    model.load_state_dict(state['model']);ema.load_state_dict(state['ema'])
    opt.load_state_dict(state['optimizer']);sched.load_state_dict(state['scheduler'])
    start_epoch,best,elapsed=state['epoch'],state['best'],state['elapsed']
    torch.set_rng_state(state['rng_torch']);np.random.set_state(state['rng_numpy'])
    random.setstate(state['rng_python']);generator.set_state(state['rng_loader'])
    event(folder,kind='resume',epoch=start_epoch)
for epoch in range(start_epoch+1,tr['epochs']+1):
    start=time.perf_counter();model.train();total=0.
    for indices,y in loader:
        views=torch.randint(bank['features'].shape[0],(len(indices),))
        x=bank['features'][views,indices]
        lam=float(np.random.beta(tr['mixup_alpha'],tr['mixup_alpha']))
        order=torch.randperm(len(x));x=lam*x+(1-lam)*x[order]
        opt.zero_grad(set_to_none=True)
        z=model(x);loss=lam*loss_fn(z,y)+(1-lam)*loss_fn(z,y[order])
        loss.backward();torch.nn.utils.clip_grad_norm_(model.parameters(),1.);opt.step()
        total+=float(loss.detach())*len(y)
    with torch.no_grad():
        for target,source in zip(ema.parameters(),model.parameters()):
            target.lerp_(source,1-tr['ema_decay'])
    vz,vy,_=predict(ema,TensorDataset(banks['val']['features'][0],banks['val']['labels']),64)
    score=balanced_nll(vz,vy);elapsed+=time.perf_counter()-start
    if score<best:
        best=score
        torch_save_atomic({'model':ema.state_dict(),'epoch':epoch,'validation_balanced_nll':best},
folder/'best.pt')
    sched.step()
    torch_save_atomic({'model':model.state_dict(),'ema':ema.state_dict(),'optimizer':opt.state_dict(),
        'scheduler':sched.state_dict(),'epoch':epoch,'best':best,'elapsed':elapsed,
        'rng_torch':torch.get_rng_state(),'rng_numpy':np.random.get_state(),
        'rng_python':random.getstate(),'rng_loader':generator.get_state()},folder/'last.pt')
    event(folder,kind='epoch',epoch=epoch,train_loss=total/n,validation_balanced_nll=score,seconds=
elapsed)
    if epoch%4==0:
        print(kernel,seed,'epoch',epoch,'validation balanced NLL',round(score,4),'seconds',round(
elapsed,1),flush=True)
    state=torch.load(folder/'best.pt',weights_only=True);model.load_state_dict(state['model'])
    vz,vy,_=predict(model,TensorDataset(banks['val']['features'][0],banks['val']['labels']),64)
    temperature=fit_temperature(vz,vy)
    meta=json.loads((CACHE/'metadata.json').read_text())
    details={'kernel':kernel,'seed':seed,'best_epoch':state['epoch'],'training_seconds':elapsed,
        'trainable_parameters':sum(p.numel() for p in model.parameters()),
        'frozen_encoder_parameters':meta['frozen_encoder_parameters'],'external_pretraining':'ImageNet-1K',
        'physical_hardware':False,'shots':0,'pennylane_qnode_calls':0,
        'head_backend':'quantum_gate_statevector_simulation' if kernel!='dot' else 'classical_dot'}
    result=None
    for split,stem in [('val','validation'),('test','test')]:
        result=save_predictions(folder,stem,model,banks[split]['features'][0],banks[split]['labels'],
temperature,details)
    print('Completed pretrained',kernel,seed,'test accuracy',round(result['accuracy'],5),flush=True)
    return result

@threadpool_limits.wrap(limits=4)
def linear_control(spec,banks):
    folder=ROOT/'linear';folder.mkdir(parents=True,exist_ok=True)
    if (folder/'test_metrics.json').exists():
        return json.loads((folder/'test_metrics.json').read_text())
    x=banks['train']['features'].mean(2).reshape(-1,512).numpy()
    y=banks['train']['labels'].repeat(banks['train']['features'].shape[0]).numpy()
    vx=banks['val']['features'][0].mean(1).numpy();vy=banks['val']['labels'].numpy()
    choices=[];models=[]
    for c in spec['linear_C_candidates']:
        model=LogisticRegression(C=c,class_weight='balanced',max_iter=2000,solver='lbfgs',tol=1e-6)
        model.fit(x,y);s=model.decision_function(vx)
        z=np.stack([-0.5*s,.5*s],1).astype(np.float32)
        choices.append({'C':c,'validation_balanced_nll':balanced_nll(z,vy),'iterations':int(model.n_iter_
[0])})
    models.append(model)
    chosen=int(np.argmin([r['validation_balanced_nll'] for r in choices]));model=models[chosen]

```

```

s=model.decision_function(vx);vz=np.stack([-0.5*s,.5*s],1).astype(np.float32)
temperature=fit_temperature(vz,vy)
np.savez_compressed(folder/'coefficients.npz',coef=model.coef_,intercept=model.intercept_)
json_write(folder/'selection.json',{ 'candidates':choices, 'selected':choices[chosen],
    'criterion':'validation_class_balanced_NLL', 'test_labels_used_for_selection':False})
for split,stem in [('val','validation'),('test','test')]:
    s=model.decision_function(banks[split]['features'][0].mean(1).numpy())
    z=np.stack([-0.5*s,.5*s],1).astype(np.float32)
    y=banks[split]['labels'].numpy();p=probabilities(z,temperature)
    np.savez_compressed(folder/(stem+'_predictions.npz'),logits=z,probabilities=p,labels=y,sample_ids=
np.arange(len(y)))
    metrics=classification_metrics(y,p)
    metrics.update(kernel='linear',C=choices[chosen]['C'],temperature=temperature,
        parameters=513,external_pretraining='ImageNet-1K',physical_hardware=False)
    json_write(folder/(stem+'_metrics.json'),metrics)
print('Pretrained linear control test accuracy',metrics['accuracy'],flush=True)
return metrics

def main():
    parser=argparse.ArgumentParser();parser.add_argument('action',choices=['freeze','run']);a=parser.
    parse_args()
    spec=json.loads(MANIFEST.read_text());frozen=MANIFEST.with_suffix('.frozen.json')
    if a.action=='freeze':
        if frozen.exists():
            raise FileExistsError(frozen)
        meta=json.loads((CACHE/'metadata.json').read_text())
        assert meta['manifest_sha256']==digest(MANIFEST) and meta['files_sha256']==feature_hashes()
        json_write(frozen,{ 'utc':datetime.now(timezone.utc).isoformat(), 'manifest_sha256':digest(MANIFEST),
            'source_sha256':source_hashes(), 'feature_metadata_sha256':digest(CACHE/'metadata.json'),
            'feature_files_sha256':feature_hashes(), 'weights_sha256':digest(spec['backbone']['weight_file'
        ]),
            'disclosure':spec['disclosure']})
        snapshot(ROOT/'frozen_source');print('Frozen pretrained quantum experiment',flush=True)
        return
    verify_lock(spec);seed_all(0,spec['training']['threads']);banks=load_features();rows=[]
    for kernel in spec['kernels']:
        for seed in spec['seeds']:
            rows.append(run_one(spec,banks,kernel,seed));json_write(ROOT/'progress.json',{'rows':rows})
    linear=linear_control(spec,banks)
    json_write(ROOT/'summary.json',{'complete':True, 'neural_conditions':len(rows), 'rows':rows, 'linear':
        linear})

if __name__=='__main__':
    main()

```

A.4 Native-resolution adapter and correspondence checks

```

"""Reuse the frozen pretrained learner under a native-resolution namespace.

No earlier computational source is edited. This explicit adapter substitutes
only the dataset resolution, manifest and artifact directories. Its own source
hash is included in the new freeze, and all matched controls share the adapter.
"""
import argparse
from dataclasses import replace
import hashlib
import json
from pathlib import Path
import numpy as np
import torch
from .data import load_splits as original_load_splits
from . import quantum_transfer as features
from . import quantum_transfer_experiment as experiment
from . import quantum_transfer_analyze as analysis
from .quantum_robust_experiment import digest
from .runtime import json_write

MANIFEST=Path('configs/native_quantum_accuracy.json')

```

```

CACHE=Path('artifacts/native_quantum_features')
ROOT=Path('runs/native_quantum_accuracy')
OUT=Path('artifacts/native_quantum_accuracy')
BASE_SOURCE_HASHES=experiment.source_hashes

def source_hashes():
    return BASE_SOURCE_HASHES()|{'research/quantum_native_accuracy.py':digest(__file__)}

def native_splits(c,data_dir='data'):
    spec=json.loads(MANIFEST.read_text())
    path=Path(spec['dataset_root'])/(c.dataset+'.npz')
    assert hashlib.md5(path.read_bytes()).hexdigest()==spec['official_md5']
    assert c.fraction==1.,'This resolution study uses complete official splits'
    datasets,channels,classes,meta=original_load_splits(replace(c,image_size=224,patch_size=112),spec['dataset_root'])
    alignment={}
    with np.load(Path('data')/(c.dataset+'.npz')) as reference:
        for split,ds in datasets.items():
            np.testing.assert_array_equal(ds.tensors[1].numpy(),reference[split+'_labels'].reshape(-1))
            reduced=torch.nn.functional.interpolate(ds.tensors[0],(28,28),mode='area').flatten(1)
            old=torch.from_numpy(reference[split+'_images'].copy()).float().flatten(1)
            reduced=reduced-reduced.mean(1,keepdim=True);old=old-old.mean(1,keepdim=True)
            denom=reduced.norm(dim=1)*old.norm(dim=1)
            valid=denom>1e-8
            corr=((reduced*old).sum(1)/denom.clamp_min(1e-8))[valid]
            fraction=float((corr>.9).float().mean())
            assert fraction>.99,(split,'Unexpected image-order correspondence',fraction)
            alignment[split]={'cases':len(ds),'labels_identical':True,'same_index_images_compared':True,
                              'nonconstant_images':len(corr),'pearson_min':float(corr.min()),
                              'pearson_median':float(corr.median()),'fraction_correlation_above_point9':fraction}
    meta.update(native_image_size=224,official_md5=spec['official_md5'],
                reference_28_sha256=digest('data/pneumoniarnist.npz'),resolution_alignment=alignment)
    CACHE.mkdir(parents=True,exist_ok=True)
    json_write(CACHE/'resolution_alignment.json',meta)
    return datasets,channels,classes,meta

def configure():
    features.MANIFEST=MANIFEST;features.CACHE=CACHE;features.load_splits=native_splits
    experiment.MANIFEST=MANIFEST;experiment.CACHE=CACHE;experiment.ROOT=ROOT
    experiment.source_hashes=source_hashes
    analysis.MANIFEST=MANIFEST;analysis.CACHE=CACHE;analysis.ROOT=ROOT;analysis.OUT=OUT
    analysis.REPORT_FILE=Path('NATIVE_QUANTUM_ACCURACY_RESULTS.md')
    analysis.source_hashes=source_hashes;analysis.load_splits=native_splits

def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('action',choices=['prepare','freeze','run','analyze'])
    parser.add_argument('--kernel',choices=['entangled','product','dot'])
    a=parser.parse_args();configure()
    if a.action=='prepare':
        features.prepare()
    elif a.action=='analyze':
        import sys
        sys.argv=[sys.argv[0],'--verify-circuits']
        analysis.main()
    elif a.action=='freeze':
        import sys
        sys.argv=[sys.argv[0],'freeze'];experiment.main()
    else:
        spec=json.loads(MANIFEST.read_text());experiment.verify_lock(spec)
        banks=features.load_features();rows=[]
        for kernel in ([a.kernel] if a.kernel else spec['kernels']):
            for seed in spec['seeds']:
                rows.append(experiment.run_one(spec,banks,kernel,seed))
                json_write(ROOT/f"progress-{a.kernel or 'all'}.json",{ 'rows':rows})
            linear=experiment.linear_control(spec,banks) if a.kernel in [None,'dot'] else None
            if a.kernel is None:
                json_write(ROOT/'summary.json',{ 'complete':True,'neural_conditions':len(rows),'rows':rows,
                'linear':linear})

```

```
if __name__ == '__main__':  
    main()
```

A.5 Frozen experiment configuration

```
{  
  "dataset": "pneumoniarnist",  
  "native_image_size": 224,  
  "dataset_root": "data/native_224",  
  "official_md5": "d6a3c71de1b945ea11211b03746c1fe1",  
  "seeds": [0, 1, 2, 3, 4],  
  "kernels": ["entangled", "product", "dot"],  
  "backbone": {"architecture": "resnet18", "weights": "IMAGENET1K_V1", "weight_file": "pretrained_models/resnet18-f37072fd.pth", "pool_size": 2, "feature_batch_size": 32, "threads": 8, "training_views": ["identity", "horizontal_flip"]},  
  "head": {"image_size": 28, "patch_size": 14, "embed_dim": 32, "heads": 2, "qk_dim": 4, "depth": 2, "mlp_dim": 64, "dropout": 0.1, "beta": 2.0},  
  "head_grid_note": "Legacy head image_size/patch_size fields set four positional tokens only; its patch embedder is deleted. The actual image encoder input is native 224 pixels.",  
  "training": {"epochs": 16, "batch_size": 64, "lr": 0.0008, "weight_decay": 0.001, "threads": 4, "warmup_epochs": 2, "ema_decay": 0.95, "mixup_alpha": 0.1, "class_weight": true},  
  "linear_C_candidates": [0.01, 0.1, 1.0, 10.0],  
  "selection": "Reuse the fixed pretrained branch recipe for all neural kernels. Best EMA checkpoint by class-balanced validation NLL. Linear regularization by the same validation criterion. No test-selected settings.",  
  "execution": "Native-224 official PneumoniaMNIST with frozen ImageNet ResNet-18, four spatial tokens, four-qubit quantum gate attention. Independent full-test PennyLane compute-uncompute verification for both quantum maps.",  
  "disclosure": "Exploratory separate resolution regime, sharing the previously inspected official cohort. External ImageNet pretraining. Higher-resolution inputs change the data regime; a gain over 28-pixel results cannot be attributed solely to quantum algorithms. All failed branches are preserved. No physical QPU or algorithmic novelty claim."  
}
```

Bibliography

- [1] MedMNIST team. MedMNIST+: standardized biomedical classification datasets with multiple size options. Official dataset release, Zenodo record 10519652. <https://zenodo.org/records/10519652>. Accessed 2 October 2026.
- [2] PyTorch torchvision. ResNet-18 model and ImageNet weight preprocessing documentation. <https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html>. Accessed 2 October 2026.
- [3] A. Mari, T. R. Bromley, J. Izaac, M. Schuld, and N. Killoran. Transfer learning in hybrid classical–quantum neural networks. *Quantum* 4, 340 (2020). <https://arxiv.org/abs/1912.08278>.
- [4] A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, and J. I. Latorre. Data re-uploading for a universal quantum classifier. *Quantum* 4, 226 (2020). <https://arxiv.org/abs/1907.02085>.